To build a long-form **Creatorize** clone from scratch using the structural mechanics, system prompts, multi-agent workflows, and Three.js rendering techniques extracted from Kristoff's architecture, follow the blueprint below.
This guide covers:

1. **The System Prompt & Core Orchestration Kernel (SKILL.md)**
2. **The Multi-Agent "Film Crew" Architecture & Node Workflow**
3. **The Three.js Deterministic Retro Rendering Engine (High-Bit Pixel Art / Voxel Spec)**
4. **Local Waveform Audio Alignment & Silence Trimming Pipeline**

1. The Core System Prompt & Agent Skill (SKILL.md)

When instantiating Claude Code or Codex locally to manage your Creatorize build core, load this skill configuration. It instructs the agent on modular file partitioning, headless rendering, and programmatic layout validation.

```
Markdown
# SKILL: Creatorize Long-Form Engine (v0.6)

## Overview
You are the Lead Director Agent for Creatorize, a local-first desktop application that orchestrates multi-agent crews to produce 10- to 30+ minute documentary videos. You coordinate the Set Designer, Scene Builders, Asset Scouts, and QA Checkers without relying on paid cloud video APIs, utilizing local Claude/Codex subscriptions, Three.js, Whisper, and FFmpeg.

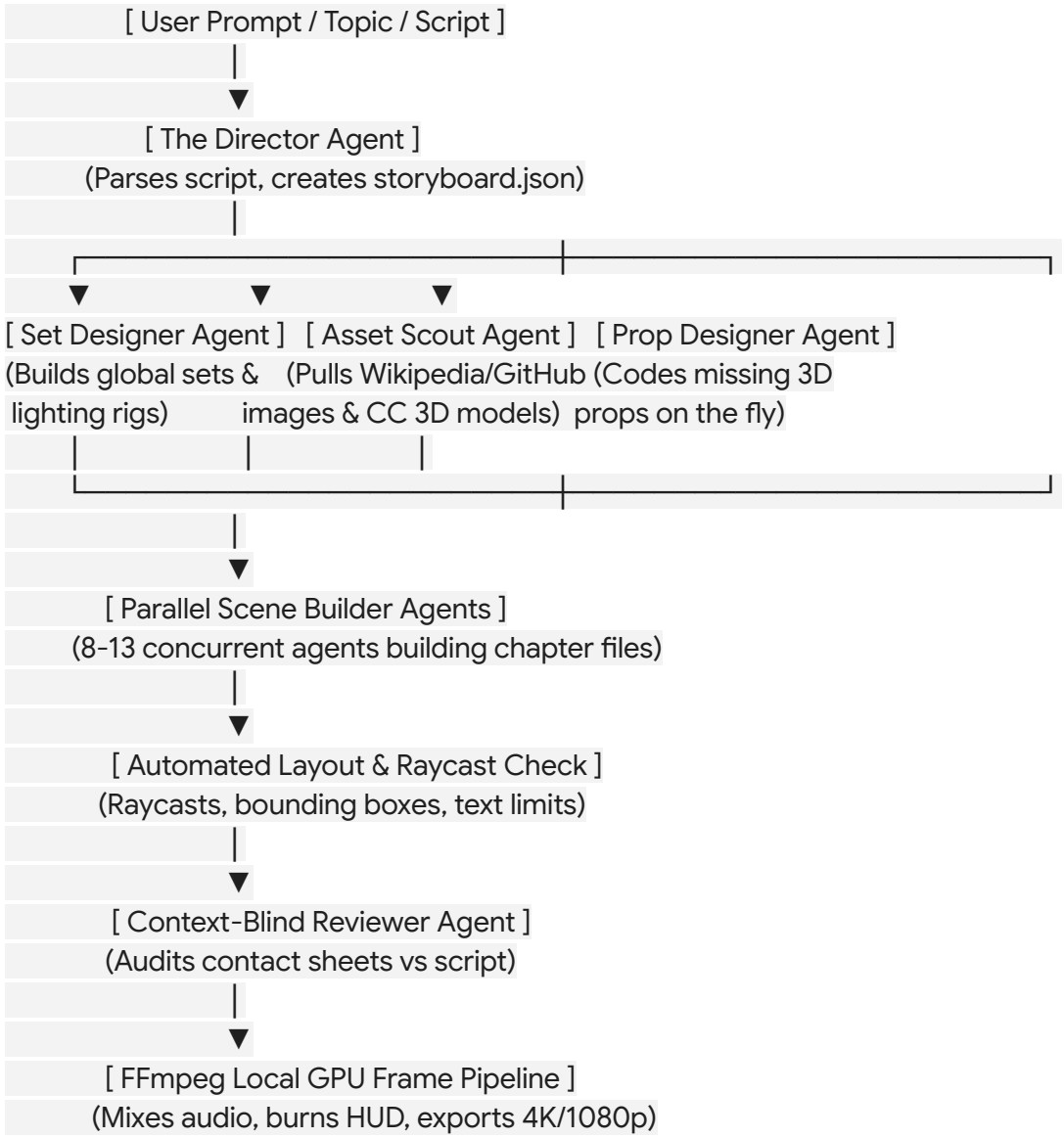
## Core Directives
1. LOCAL INFRASTRUCTURE: Run everything via local CLI sessions. Pipe headless browser frames directly to FFmpeg.
2. PARALLEL CREW ISOLATION: Never write a long video in a single monolithic file. Split long-form videos by chapters (max 70s per file: `chapter_01.js`, `chapter_02.js`), utilizing strict file isolation so concurrent builder agents never overwrite each other.
3. DETERMINISTIC RENDERING ($f(t)$): All scenes in Three.js must be pure deterministic functions of time. No procedural jitter, random wandering, or unkeyed asset drift.
4. AUTOMATED QA GATES: Before handing a chapter back to the director, execute programmatic bounding-box and raycast checks to catch overlapping text, clipped characters, or empty frames.

## Pipeline Steps
- STEP 1: Parse word-level transcript timings from Whisper + numpy/scipy noise-gate alignment.
- STEP 2: The Director writes `storyboard.json`, `facts.md`, and `broll.md`.
- STEP 3: Launch Set Designer & Parallel Scene Builders simultaneously.
```

- STEP 4: Run automated layout check and context-blind review agent.
- STEP 5: Mix audio layers (-14 LUFS master, music bed at -18 LU under voice) and export via FFmpeg.

2. Multi-Agent "Film Crew" Orchestration Workflow

To prevent context-window exhaustion and cut build times down to ~30 minutes for a 26-minute video, implement this orchestration flow inside your Node.js/Electron main process.



Node.js / TypeScript Orchestrator Stub

```typescript
import { spawn } from 'child_process';
import fs from 'fs/promises';

interface CrewConfig {
  chapterFile: string;
  duration: number;
  style: string;
}

async function launchParallelCrew(chapters: CrewConfig[]) {
  console.log(`[Director] Launching Set Designer and ${chapters.length} Scene Builders...`);

  // Launch Set Designer first
  const setDesigner = spawn('node', ['agents/set-designer.js', '--init'], { stdio: 'inherit' });
  await new Promise((resolve) => setDesigner.on('close', resolve));

  // Launch Asset Scout in background
  spawn('node', ['agents/asset-scout.js', '--auto-fetch'], { detached: true });

  // Launch Parallel Builders
  const buildPromises = chapters.map((chap) => {
    return new Promise((resolve, reject) => {
      const builder = spawn('node', ['agents/scene-builder.js', '--file', chap.chapterFile], { stdio:
'inherit' });
      builder.on('close', (code) => {
        if (code === 0) resolve(true);
        else reject(new Error(`Builder failed on ${chap.chapterFile}`));
      });
    });
  });

  await Promise.all(buildPromises);
  console.log('[Director] All chapters built successfully. Running QA Layout Checks...');
}
```

3. Three.js Deterministic Retro Rendering Engine (High-Bit Pixel Art Spec)

Creatorize's signature look uses a Three.js scene rendered at a low internal resolution (640x360) with custom vertex-snapping/affine-warping shaders to mimic retro hardware limits, then upscaled with hard pixels to 1080p/4K.

Three.js Retro Scene Setup (renderer.js)

```javascript
import * as THREE from 'three';

export function createRetroEngine(containerId, width = 640, height = 360) {
 const scene = new THREE.Scene();
 scene.background = new THREE.Color(0x111118);

 const camera = new THREE.PerspectiveCamera(45, width / height, 0.1, 1000);
 camera.position.set(0, 5, 12);
 camera.lookAt(0, 0, 0);

 // Low-res render target for retro pixel look
 const renderTarget = new THREE.WebGLRenderTarget(width, height, {
  minFilter: THREE.NearestFilter,
  magFilter: THREE.NearestFilter,
  format: THREE.RGBAFormat,
 });

 const renderer = new THREE.WebGLRenderer({ antialias: false });
 renderer.setSize(window.innerWidth, window.innerHeight);
 document.getElementById(containerId).appendChild(renderer.domElement);

 // Custom Retro Vertex Shader (Vertex Snapping & Affine Texture Warping)
 const retroMaterial = new THREE.MeshStandardMaterial({
  color: 0x3399ff,
  roughness: 0.8,
  metalness: 0.2,
 });

 return { scene, camera, renderer, renderTarget, retroMaterial };
}

// Deterministic frame update loop function f(t)
export function updateSceneTime(mesh, timeInSeconds) {
```

```
    // Hard snap position to grid step to simulate retro hardware floating-point limits
    const gridStep = 0.05;
    mesh.position.x = Math.round(Math.sin(timeInSeconds) * 3 / gridStep) * gridStep;
    mesh.rotation.y = Math.floor(timeInSeconds * 30) / 30; // 30fps animation hold simulation
}
```

4. Local Waveform Audio Alignment & Silence Trimming Pipeline

To guarantee that 3D camera movements and text cards switch exactly on the spoken word without drift, the system aligns local Whisper outputs against python/scipy audio noise gates.

Python Alignment Utility (align_audio.py)

```python
import sys
import json
import numpy as np
from scipy.io import wavfile
import whisper

def align_waveform(wav_path, model_size="turbo"):
    sample_rate, data = wavfile.read(wav_path)
    if len(data.shape) > 1:
        data = data.mean(axis=1) # Convert to mono

    # Simple Noise Gate to detect actual speech segments
    abs_data = np.abs(data)
    threshold = np.max(abs_data) * 0.05
    speech_mask = abs_data > threshold

    # Load local Whisper model
    model = whisper.load_model(model_size)
    result = model.transcribe(wav_path, word_timestamps=True)

    aligned_words = []
    for segment in result["segments"]:
        for word in segment["words"]:
            aligned_words.append({
                "word": word["word"].strip(),
                "start": round(word["start"], 3),
                "end": round(word["end"], 3)
```

```
        })

    output_json = {"sample_rate": sample_rate, "words": aligned_words}
    print(json.dumps(output_json))

if __name__ == "__main__":
    align_waveform(sys.argv[1])
```